LAB 8

Task 1:

Code	Explanation

[org 0x0100]
jmp start
 	
num: db 11100101b
start:
  mov al, num
  rol al, 1
  ror al, 1
mov ax, 0x4c00
int 0x21
	1. Start.
	2. saved 8 bit number 11100101b
	3. moved the number to al register
	4. rotate left result (11001010b) CF = 1.
	5. Rotate right result the same number
	6. Exit program

Task 2:

Code	Explanation

[org 0x0100]
jmp start
multiplicand: db 4
multiplier: db 3
result: db 0
 	
start:
  mov cl, 4
  mov bl, [multiplicand]
  mov dl, [multiplier]
 	
  checkbit:
  shr dl, 1
  jnc skip
  add [result], bl
  skip:
  shl bl, 1
  dec cl
  jnz checkbit
mov ax, 0x4c00
int 0x21
	❖ Start.
	❖ store 4 in multipicand label
	❖ store 3 in multiplier label
	❖ store 0 in result label.
	❖ mov 4 in cl register
	❖ mov the value of multiplicand into bl register which is 4.
	❖ mov the value of multiplier into dl register which is 3.
	❖ run loop 4 times.
	❖ each time shift right the value of dl by 1.
	❖ check if carry is 0 if it is then jump to skip label else add value of bl into result.
	❖ then shift left bl by 1
	❖ repeat the process

Task 3:

Code	Explanation

[org 0x0100]
jmp start
num1: dw 000Ah
num2: dw 0003h
high: dw 0000h
low: dw 0000h
 	
num3: dw 00A0h
num4: dw 000Ah
s_high: dw 0000h
s_low: dw 0000h
start:
clc
mov ax, [num1]
add ax, [num2]
mov word [low], ax
adc word [high], 0
 	
mov ax, [num3]
sub ax, [num4]
mov word [s_low], ax
sbb word [s_high], 0
mov ax, 0x4c00
int 0x21
	❖ Initialize some important labels for addtion and subtraction…num1, num2 , high and low for extended addition.
	❖ num3, num4, s_high and s_low for extended subtraction.
	❖ after start label first clear the carry to 0.
	❖ mov value of num1 into ax register.
	❖ add value of num2 into ax register.
	❖ mov value of qx into low.
	❖ add carry into high
	
	For subtraction:
	❖ mov num3 into ax
	❖ subtract num3 from ax and store into ax.
	❖ mov value of ax into s_low.
	❖ subtract bit
	❖ end program
	❖

Task 4:

Code	Explantion

[org 0x0100]
jmp start
num1: db 00A0h
num2: db 00BBh
result: db 0
start:
mov al, [num1]
mov bl, [num2]
 	
and al, bl
mov [result], al
mov al, [num1]
 	
or al, bl
mov [result], al
mov al, [num1]
	❖ start
	❖ store some values in num1 and num2.
	❖ result = 0.
	❖ store num1 in al register and num2 in bl register.
	❖ Perform AND operation. give 1 only when both bits are 1. mov the result into result label.
	❖ Now again mov num1 into al register.
	❖ Perform OR operation, gives 1 when either al bit is 1 or bl bit is 1.
	❖ mov result into result.
	❖ again mov num1 into al.
	❖ Perform xor operation. gives 1 only if both values are different.
	❖ store result into result.

xor al, bl
mov [result], al
mov al, [num1]
 	
not al
mov [result], al
mov ax, 0x4c00
int 0x21
	❖ mov num1 into al.
	❖ perform not operation into al
	❖ mov result into result